Department of Electronic & Telecommunication Engineering
University of Moratuwa

UART Implementation in FPGA

EN2111 Electronic Circuit Design

Group 26

PERERA H.A.J.I	230477X
PERERA K.W.A.O.V	230481E
PERERA U.I.H	230486A

May 1, 2026

Contents

1 Introduction **2**

2 UART Protocol **2**
2.1 Frame Format 2
2.2 Baud Rate Calculation 2

3 System Overview **2**
3.1 Block Diagram 2
3.2 Hardware I/O Mapping 4

4 RTL Design **4**
4.1 UART Transmitter (`uart_tx`) 4
4.2 UART Receiver (`uart_rx`) 4
4.2.1 3-Stage Synchronizer 4
4.2.2 Mid-Bit Sampling 4
4.3 7-Segment Decoder (`seven_seg`) 5
4.4 Top-Level Module (`uart_top`) 5

5 RTL Code **5**

6 Testbench Design **5**
6.1 Test Patterns 6
6.2 Testbench Procedure 6

7 Simulation Results **6**
7.1 Observations 7
7.2 Discussion 8

8 FPGA Implementation **8**
8.1 Hardware Setup 8
8.2 Verification with Another Group 9

9 Oscilloscope Verification **9**
9.1 Observations 9

10 Conclusion **9**

A GitHub Repository **10**

1 Introduction

Universal Asynchronous Receiver Transmitter (UART) is one of the most widely used serial communication protocols in embedded systems and digital hardware design. Unlike synchronous protocols, UART does not require a shared clock signal between the communicating devices. Instead, both the transmitter and receiver agree on a fixed baud rate beforehand, allowing them to independently time the sampling of each data bit.

In this assignment, a complete UART transceiver was designed and implemented using Verilog HDL and deployed on an Intel DE0-Nano FPGA development board. The system transmits a user-selected byte (entered via DIP switches) over a serial line and displays the received value on 7-segment displays. The goal was to understand UART protocol internals, practice FSM-based RTL design, and validate the design through simulation and physical hardware testing.

2 UART Protocol

2.1 Frame Format

UART transmits data one bit at a time, encapsulated within a fixed-length frame. The frame format used in this implementation is:

Start	**D0**	**D1**	**D2**	**D3**	**D4**	**D5**	**D6**	**D7**	**Stop**
0	LSB							MSB	1

- **Start bit**: Logic 0 — signals the beginning of a frame.
- **Data bits (D0–D7)**: 8 bits transmitted LSB-first.
- **Stop bit**: Logic 1 — signals the end of the frame and returns the line to idle.

The line is held at logic 1 (idle/mark state) when no transmission is occurring. Each bit is held for a fixed duration determined by the baud rate.

2.2 Baud Rate Calculation

The DE0-Nano provides a 50 MHz on-board clock. The baud rate is set to 115200 bps, requiring:

$$\text{CLKS_PER_BIT} = \frac{f_{clk}}{\text{Baud Rate}} = \frac{50{,}000{,}000}{115{,}200} \approx 434$$

This parameter is used in both the transmitter and receiver to count clock cycles per bit period. Each bit is therefore held for exactly 434 clock cycles ($\approx 8.68\,\mu$s).

3 System Overview

3.1 Block Diagram

The overall system is composed of the following Verilog modules:

Table 1: RTL Modules

Module	Purpose
`uart_tx`	Converts parallel 8-bit data into a serial UART bitstream
`uart_rx`	Reconstructs received serial bits back into parallel 8-bit data
`seven_seg`	Drives a 7-segment display from a 4-bit hexadecimal input (active-low)
`uart_top`	Top-level wrapper: maps FPGA pins, edge-detects button, instances TX/RX/7-seg
`uart_tb`	Simulation testbench; loops through 6 test patterns and reports PASS/-FAIL

The system block diagram is shown in Figure 1.

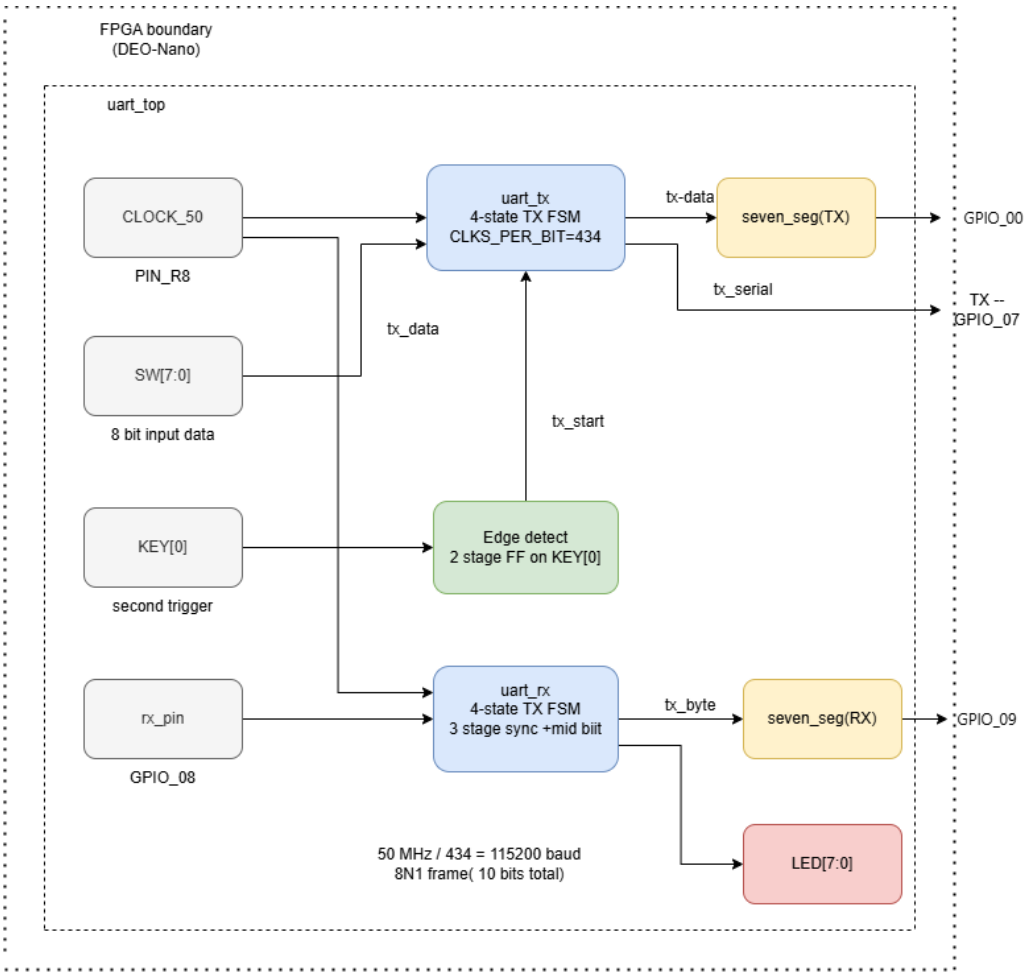

Figure 1: System Block Diagram

3.2 Hardware I/O Mapping

Table 2: FPGA Pin Assignment

Signal	FPGA Pin	Function
CLOCK_50	PIN_R8	50 MHz system clock
KEY[0]	PIN_J15	Active-low send trigger (falling-edge detected)
SW[7:0]	PIN_C9 – PIN_D11	8-bit input data
GPIO_07	TX pin	UART serial output
GPIO_08	RX pin	UART serial input
LED[7:0]	PIN_L3 – PIN_A15	Displays received byte on LEDs
GPIO_00--06	—	TX 7-segment display segments
GPIO_09--15	—	RX 7-segment display segments

4 RTL Design

4.1 UART Transmitter (uart_tx)

The transmitter is implemented as a 4-state Mealy FSM. Once tx_start is asserted for one clock cycle, the input byte is latched and the FSM sequences through the UART frame.

- **IDLE**: Line held high. Waits for tx_start. Latches tx_data and moves to START.
- **START**: Drives line low for exactly CLKS_PER_BIT cycles, then moves to DATA.
- **DATA**: Outputs bits data_reg[bit_idx] (LSB first). Advances bit_idx after each bit period. After bit 7, moves to STOP.
- **STOP**: Drives line high for one bit period, then returns to IDLE.

4.2 UART Receiver (uart_rx)

The receiver mirrors the transmitter FSM but adds two important reliability features:

4.2.1 3-Stage Synchronizer

The raw rx_serial pin is passed through a three-stage flip-flop chain before use. This eliminates metastability caused by the asynchronous incoming signal entering the FPGA clock domain.

```
reg [2:0] rx_sync = 3'b111;
always @(posedge clk) rx_sync <= {rx_sync[1:0], rx_serial};
wire rx_clean = rx_sync[2];
```

Listing 1: 3-stage synchroniser in receiver

4.2.2 Mid-Bit Sampling

Rather than sampling immediately when the start bit edge is detected, the receiver waits for half a bit period ((CLKS_PER_BIT-1)/2 cycles) to confirm the start bit is genuine and

not a glitch, then resets its counter. After that, it samples each subsequent bit in the middle of its bit period, which is the point of maximum signal stability.

The 4-state RX FSM operates as follows:

- **IDLE**: Waits for `rx_clean` to go low (start bit edge detected).
- **START**: Counts to mid-bit. If still low, proceeds to DATA; otherwise returns to IDLE (glitch rejection).
- **DATA**: Samples `rx_clean` into `data_reg[bit_idx]` at each full bit period boundary. After 8 bits, moves to STOP.
- **STOP**: Waits one bit period, then asserts `rx_dv` (data valid) for one cycle and outputs `rx_data`.

4.3 7-Segment Decoder (`seven_seg`)

The decoder converts a 4-bit hexadecimal value (0x0–0xF) to a 7-bit active-low segment pattern. The received byte is split into its upper and lower nibbles (`rx_byte[7:4]` and `rx_byte[3:0]`), each driven to a separate 7-segment display.

4.4 Top-Level Module (`uart_top`)

The top module wires all sub-modules together and handles button edge detection. `KEY[0]` is active-low on the DE0-Nano; a falling edge (high-to-low transition) triggers transmission. A two-stage synchroniser on the key input ensures stable edge detection:

```
always @(posedge clk) begin
    key0_d1 <= key0_input;
    key0_d2 <= key0_d1;
end
assign tx_pulse = (key0_d2 == 1'b1 && key0_d1 == 1'b0);
```

Listing 2:]Falling-edge detection on KEY[0]

`tx_pulse` is asserted for exactly one clock cycle per button press, ensuring a single byte is transmitted per press regardless of how long the button is held.

5 RTL Code

The complete Verilog source files are available in the project GitHub repository:

https://github.com/OshadhaVihanga/uart-fpga-verilog

6 Testbench Design

A self-checking Verilog testbench (`uart_tb`) was written to verify the full TX→RX path in simulation without requiring any manual inspection. The TX serial output is connected directly to the RX serial input, forming a loopback.

6.1 Test Patterns

Six byte patterns were chosen to exercise a wide range of bit combinations:

Table 3: Testbench Patterns

Index	Hex	Binary	Intent
0	`0xA5`	`10100101`	Alternating bits
1	`0x3C`	`00111100`	Mid-byte burst
2	`0xFF`	`11111111`	All ones
3	`0x00`	`00000000`	All zeros
4	`0x5A`	`01011010`	Inverted alternating
5	`0x81`	`10000001`	Boundary bits only

6.2 Testbench Procedure

1. Assert `tx_start` for one clock cycle with the test pattern loaded into `tx_data_in`.
2. Wait for `rx_valid` (`rx_dv`) to go high — this indicates the receiver has completed reception.
3. Compare `rx_data_out` against the sent pattern and print `PASS` or `FAIL`.
4. Wait two bit periods before the next transmission to ensure the line returns to idle.

The 50 MHz clock is modelled with a 20 ns period (`CLK_PERIOD = 20`), matching the physical hardware exactly.

7 Simulation Results

Simulation was performed using ModelSim. Figures 2–4 show the waveform at key stages. A screen recording of the full simulation run is included in the GitHub repository.

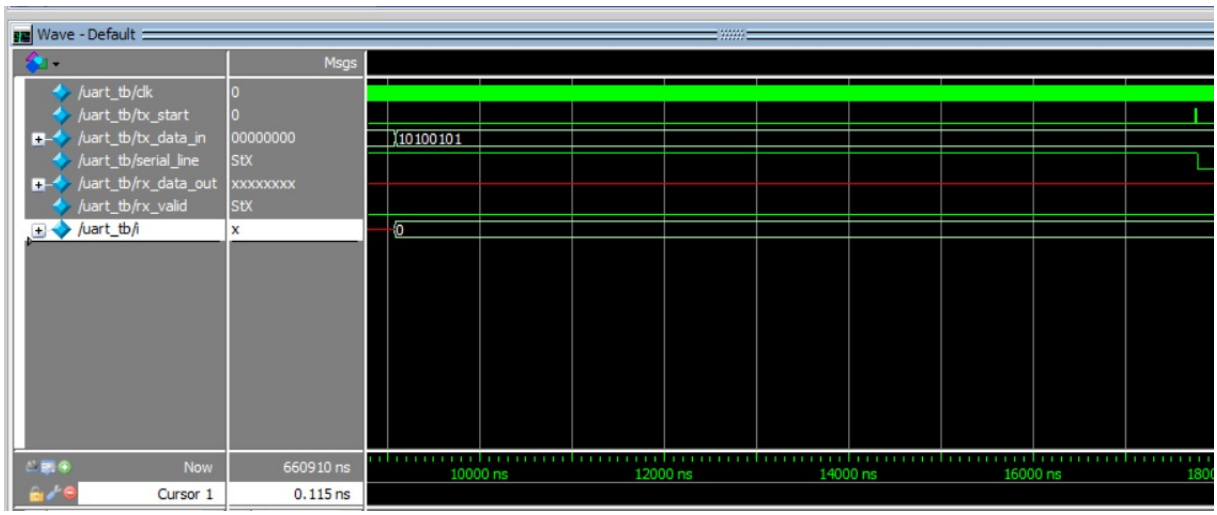

Figure 2: Simulation waveform at initialisation — TX line idle (high), all outputs at reset state.

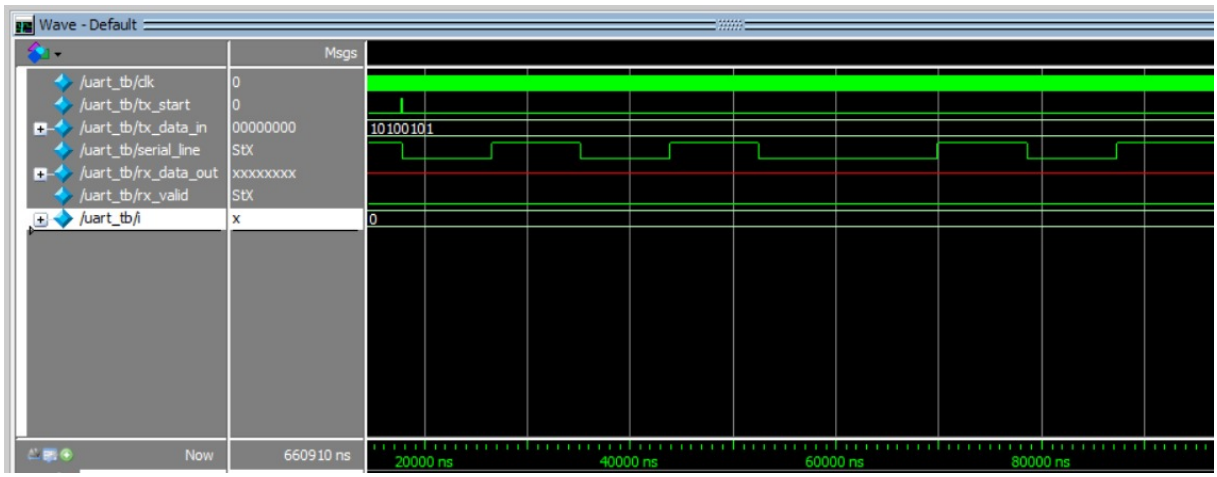

Figure 3: Waveform after the first transmission is triggered — start bit drives TX low, followed by 8 data bits.

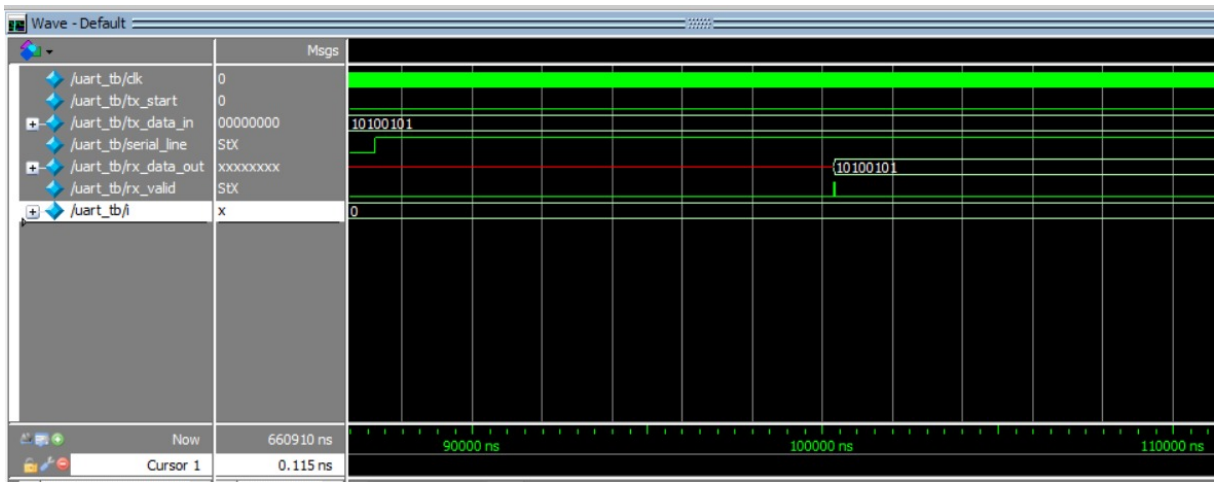

Figure 4: Waveform after reception is complete — `rx_dv` pulses high and `rx_data` matches the transmitted byte.

7.1 Observations

- The transmitter enters the START state and holds TX low for exactly 434 clock cycles before outputting data bits.
- Data bits are transmitted LSB-first, confirmed by inspecting the serial waveform against the known pattern.
- The receiver's mid-bit sampling logic correctly ignores the synchroniser pipeline delay and reconstructs each byte without error.
- All 6 test patterns produced a `PASS` result: `rx_data_out` matched `tx_data_in` in every case.
- The `rx_dv` pulse lasts exactly one clock cycle per received byte, as designed.

7.2 Discussion

The simulation confirms that the FSM timing is correct. The receiver's decision to sample at the midpoint of the bit period provides maximum noise margin — any edge jitter of less than half a bit period (i.e., less than $\sim$4.34 μs at 115200 baud) will not cause a sampling error. The 3-stage synchroniser adds a 3-cycle latency to the start-bit detection, but this is negligible ($\sim$60 ns) compared to the bit period.

The testbench patterns were specifically chosen to stress boundary conditions: `0x00` (all zeros) produces the longest possible consecutive low period on the serial line, and `0xFF` produces all high data bits following the start bit — both are common sources of framing errors in poorly designed receivers.

8 FPGA Implementation

8.1 Hardware Setup

The design was synthesised and programmed onto the DE0-Nano board using Intel Quartus Prime. Figure 5 shows the physical setup. Two DE0-Nano boards were used — TX of one board was connected to RX of the other via a GPIO wire — to demonstrate cross-board UART communication with another group.

DIP switches `SW[7:0]` select the 8-bit value to transmit. Pressing `KEY[0]` sends the byte. The received value is displayed simultaneously on the on-board LEDs and on an external 7-segment display driven through GPIO pins.

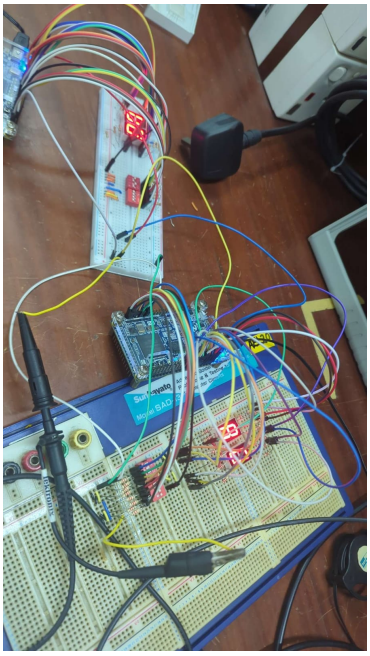

Figure 5: FPGA hardware setup showing DE0-Nano board, GPIO connections, and 7-segment displays.

8.2 Verification with Another Group

The TX output of our board was connected to the RX input of another group's board and vice versa. Both boards successfully decoded each other's transmissions, confirming that the baud rate and framing are compatible.

9 Oscilloscope Verification

The UART TX signal was probed on a Tektronix TBS 1000C digital oscilloscope. Figure 6 shows the captured waveform.

Figure 6: UART TX waveform captured on oscilloscope (200 μs/div).

9.1 Observations

- The start bit (logic 0) is clearly visible as the first falling edge, followed by 8 data bits and the stop bit (logic 1).
- At 200 μs per division, each bit occupies approximately 8.68 μs, consistent with the expected value:

$$T_{bit} = \frac{1}{115200} \approx 8.68\,\mu\text{s}$$

- The voltage levels measure approximately 3.3 V for logic 1 and 0 V for logic 0, matching the DE0-Nano's GPIO voltage standard.
- No ringing or signal integrity issues were observed, indicating clean switching.

10 Conclusion

A fully functional UART transceiver was successfully designed in Verilog HDL and implemented on the DE0-Nano FPGA. The transmitter and receiver are each implemented as 4-state FSMs clocked at 50 MHz with `CLKS_PER_BIT = 434`, corresponding to a baud rate of 115200 bps with 8 data bits, 1 start bit, and 1 stop bit (8N1 format).

Simulation using a self-checking testbench confirmed correct operation across all 6 test patterns, including edge cases such as all-zeros and all-ones payloads. Hardware testing on the DE0-Nano produced waveforms on the oscilloscope matching the expected bit timing, and cross-board verification with another group confirmed interoperability. The 7-segment displays correctly reflected the received hexadecimal values in real time.

A GitHub Repository

All source files, simulation scripts, and demonstration videos are available at:

https://github.com/OshadhaVihanga/uart-fpga-verilog

The repository includes:

- `uart_tx.v` — Transmitter RTL
- `uart_rx.v` — Receiver RTL
- `seven_seg.v` — 7-segment decoder
- `uart_top.v` — Top-level FPGA module
- `uart_tb.v` — Simulation testbench
- `simulation/` — ModelSim waveform screenshots and recording